some layers in a neural network and then learns with the reduced neural network. In this way, the network learns to be independent. It is not reliable on a single layer. This helps in overcoming overfitting. We can specify a value between 0 and 1 for the input to the dropout layer. For instance, if we select 0.5, it means to drop half of the layers randomly.

The final layer has an output size of one. It does not use the previously used activation function ReLU, but a different activation function called sigmoid. This is because we are trying to classify if an image is a dog or a cat. We want the model to output a probability of how sure an image is a dog and not a cat. It means that we want a probability score between 0 and one where higher values close to 1 means that the classifier believes the image is a dog and lower values mean it is a cat. The sigmoid activation function is perfect for this because it takes in a set of numbers and returns a probability in the range of 0 to 1. This helps in classifying the images.

To preview the arrangement and parameter size of our convolutional neural network, we call the Keras function .summary() on the model object. It shows us the number of parameters that we want to train which are more than three million in this model. It also shows the general architecture of the different layers, along with the training parameters of each layer.

Compiling our model:

After developing our architecture of the training model, our next step is to compile it. There are three parameters to the model. Compile () command – a loss function, an optimiser and metric to evaluate model performance. They are discussed as follows:

1. Loss function: The first parameter is a loss function that our optimiser will minimise. Keras provides several loss functions such as categorical_crossentropy, sparse_categorical_crossentropy, binary_crossentropy, kullback_leibler_divergence and many others. Since we are working with a two-class classifier problem in our model, we will use the binary cross-entropy loss function.
2. Next parameter is to specify one of the optimisers. We are going to use the rmsprop optimiser for this model. The choice of optimisers can vary a lot according to the complexities of different models since it is part of a process called hyper-parameter tuning. This is an important process in differentiating a good accuracy model from a naive one.
3. Finally, in the third parameter, we specify the metric that we want to use to evaluate our model's performance after training. We want to know